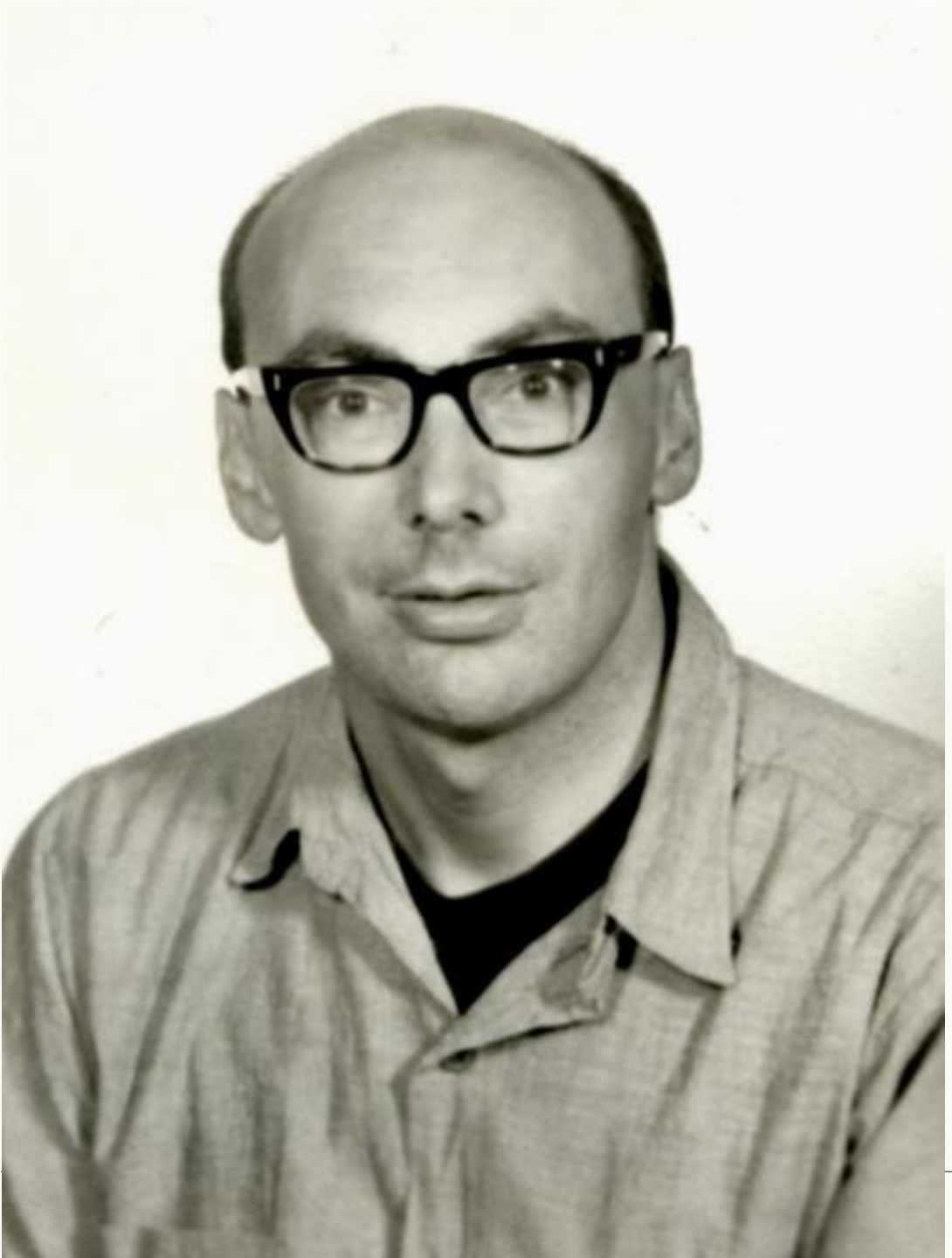


gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

DAT321 DIT843 Code Coverage

Dr. Francisco Gomes de Oliveira Neto (he / him)

Peter Landin

- **Pioneer in functional programming.** One of the first to realise that the lambda calculus could be used to model a programming language.
- Defined the precursor of the off-side rule instrumental in Python
- As a Bissexual man, he was politically involved with LGBTQIA+ causes.

What we will learn today?

- What is **Test Adequacy**? Why should we use it when planning our tests?
- What is **Structural-based Testing**? Why do we use it? How do we use it?
- What are the different **code coverage measures**? How can we measure them?
- What is **MC/DC Coverage**? Why is it important? When should we use it?
- What are subsumed criteria? What are the **strongest and weakest** criteria?

© Francisco Oliveira Neto - 2018

Test Adequacy Criteria

- Test Adequacy is a quality attribute that **conveys the extent** of the test effort. (How good our tests are?!). Based on test obligations.

- There are several ways to measure test adequacy but the most popular is code-based adequacy (e.g., structural-based coverage).
  - **Coverage** adequacy: Exercises various **code contructs** (method, branch, statements)
  - **Surprise** adequacy: Exercises the **surprising cases** to check ML software (https://arxiv.org/abs/1808.08444)
- The goal is to show that our **tests meet those criteria**.

© Francisco Oliveira Neto - 2018

The Size of the Test (and the Suite)

- Tests should help with debugging. **Balanced size** is essencial.

- **Test Suites with more targeted TCs** are typically better than fewer and larger TCs.
  - Tests should also be independent from each other.
  - Having fewer tests implies more asserts per tests, which can hinder debugging.

- However, more tests can slow down test execution.

© Francisco Oliveira Neto - 2018

Structural Testing – What, Why and How

- **What:** Use **information from the source code** to design test cases. Typically structural testing is guided by **coverage** information. There are multiple code coverage measures.

- **Why:** Knowing the code structure **helps us understand when to stop testing**. Since faults are mistakes in the code, knowing the what parts of the code the tests exercise **increases our confidence** that our tests will reveal faults.

- **How:** We use **code constructs** (statements, conditions, paths) to guide the test generation and assessing the **test adequacy**.

© Francisco Oliveira Neto - 2018

Structural Testing – Branch vs. Condition

- In structural testing, Branches, Decisions and Conditions refer to **different constructs**.

- **Branches** are **if-then** constructs:
  - E.g.: if(a > b) { … }; if(x == 10) { … } else { … }
- **Conditions** are "**atomic**" boolean expressions.
  - E.g.: (a > b) is a condition; (x == 10) is a condition; (a > b) || (x == 10) is a decision, **NOT** a condition.

- Expressions connected by **AND**, **OR** and other boolean operators are decisions, they are not seen as conditions.

© Francisco Oliveira Neto - 2018

Review of Coverage Measures

- Line coverage: $\frac{lines_covered}{total_lines} * 100.$
  - Counts the **actual line**, so may vary for coding guidelines.
- Branch coverage: $\frac{branches_cov}{total_branches} * 100.$
  - Branches are **outoing edges** from decision nodes in the CFG.
- Condition coverage: $\frac{conditions_outcome_cov}{total_conditions} * 100.$
  - Conditions are **values** within a boolean expression (decision). Does not consider the edges, instead, **only the values exercised** by each condition.

© Francisco Oliveira Neto - 2018

Review of Coverage Measures

- Path coverage: $\frac{paths_covered}{total_paths} * 100.$
  - Considers possible ways to **traverse the flow graph** instead of individual condition values.

© Francisco Oliveira Neto - 2018

Modified Condition / Decision Coverage

- MC/DC exercise each condition in a way that it can, **independently of the other conditions**, **affect** the outcome of the entire decision.

- MC/DC is a **compromise** from path coverage. We do not need all possible paths, since some condition coverage will be redundant.
- Very important for **safety critical** software (avionics, automotive, etc.)

© Francisco Oliveira Neto - 2018

Modified Condition / Decision Coverage

- MCDC requires that the **final test suite**:
  - Evaluate each **decision** to both TRUE / FALSE
  - Evaluate each **condition** to both TRUE / FALSE
  - Evaluate each condition that **independently affect** the outcome of a decision.

- **Always N + 1 tests** needed, where N is the number of conditions.

© Francisco Oliveira Neto - 2018

Steps for MC/DC Coverage

1. Build **all possible outcomes** from all combinations.
2. **Identify independence** for each condition by **varying one** condition and fixing the others.
3. **Minimise the set** of test cases that exercise the decision for each condition checked above.
   - When minimising, there may be **more than one solution**.

© Francisco Oliveira Neto - 2018

MC/DC Coverage: Example 1

- **Condition to approve the purchase of a product in an online store:**
- `if((isInStock(item) && isPaymentApproved(item)) || hasBuyerCode()){…`

isInStock(item)	isPaymentApproved(item)	hasBuyerCode()	Decision

© Francisco Oliveira Neto - 2018

MC/DC Coverage: Example 1

- **Condition to approve the purchase of a product in an online store:**
- `if((isInStock(item) && isPaymentApproved(item)) || hasBuyerCode()){…`
- Step 1: All cases

isInStock(item)	isPaymentApproved(item)	hasBuyerCode()	Decision
FALSE	FALSE	FALSE	FALSE
FALSE	FALSE	TRUE	TRUE
FALSE	TRUE	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	FALSE
TRUE	FALSE	TRUE	TRUE
TRUE	TRUE	FALSE	TRUE
TRUE	TRUE	TRUE	TRUE

© Francisco Oliveira Neto - 2018

MC/DC Coverage: Example 1

- **Condition to approve the purchase of a product in an online store:**
- `if(isInStock(item) && isPaymentApproved(item)) || hasBuyerCode()){…`
- Step 1: All cases
- Step 2: Independent conditions:
  - hasBuyerCode: {T1,T2},{T3,T4},{T5,T6}
  - isPaymentApproved:{T5,T7}
  - isInStock:{T3,T7}

	isInStock	isPaymentApproved	hasBuyerCode	Decision
T1	FALSE	FALSE	FALSE	FALSE
T2	FALSE	FALSE	TRUE	TRUE
T3	FALSE	TRUE	FALSE	FALSE
T4	FALSE	TRUE	TRUE	TRUE
T5	TRUE	FALSE	FALSE	FALSE
T6	TRUE	FALSE	TRUE	TRUE
T7	TRUE	TRUE	FALSE	TRUE
T8	TRUE	TRUE	TRUE	TRUE

© Francisco Oliveira Neto - 2018

MC/DC Coverage: Example 1

- **Condition to approve the purchase of a product in an online store:**
- `if(isInStock(item) && isPaymentApproved(item)) || hasBuyerCode()){…`
- Step 1: All cases
- Step 2: Independent conditions:
  - hasBuyerCode: {T1,T2},{T3,T4},**{T5,T6}**
  - isPaymentApproved:**{T5,T7}**
  - isInStock:**{T3,T7}**
- Step 3: The minimal set is:
  - {T3, T5, T6, T7}

	isInStock	isPaymentApproved	hasBuyerCode	Decision
T1	FALSE	FALSE	FALSE	FALSE
T2	FALSE	FALSE	TRUE	TRUE
T3	FALSE	TRUE	FALSE	FALSE
T4	FALSE	TRUE	TRUE	TRUE
T5	TRUE	FALSE	FALSE	FALSE
T6	TRUE	FALSE	TRUE	TRUE
T7	TRUE	TRUE	FALSE	TRUE
T8	TRUE	TRUE	TRUE	TRUE

© Francisco Oliveira Neto - 2018

MC/DC Coverage: Example 2

- Can vote in municipality: (isRegistered || isCitizen) && (age > 18)

isRegistered	isCitizen	age > 18	Decision

© Francisco Oliveira Neto - 2018

MC/DC Coverage: Example 2

- Can vote in municipality: (isRegistered || isCitizen) && (age >= 18)
- Step 1: All cases

Test	isRegistered	isCitizen	age >= 18	Decision
T1	FALSE	FALSE	FALSE	FALSE
T2	FALSE	FALSE	TRUE	FALSE
T3	FALSE	TRUE	FALSE	FALSE
T4	FALSE	TRUE	TRUE	TRUE
T5	TRUE	FALSE	FALSE	FALSE
T6	TRUE	FALSE	TRUE	TRUE
T7	TRUE	TRUE	FALSE	FALSE
T8	TRUE	TRUE	TRUE	TRUE

© Francisco Oliveira Neto - 2018

MC/DC Coverage: Example 2

- Can vote in municipality: (isRegistered || isCitizen) && (age >= 18)
- Step 2: Identify
- isReg: {T2,T6};
- isCit: {T2,T4};
- age: {T3,T4}; {T5,T6};

  {T7,T8}

Test	isRegistered	isCitizen	age >= 18	Decision
T1	FALSE	FALSE	FALSE	FALSE
T2	FALSE	FALSE	TRUE	FALSE
T3	FALSE	TRUE	FALSE	FALSE
T4	FALSE	TRUE	TRUE	TRUE
T5	TRUE	FALSE	FALSE	FALSE
T6	TRUE	FALSE	TRUE	TRUE
T7	TRUE	TRUE	FALSE	FALSE
T8	TRUE	TRUE	TRUE	TRUE

© Francisco Oliveira Neto - 2018

MC/DC Coverage: Example 2

- Can vote in municipality: (isRegistered || isCitizen) && (age >= 18)
- Step 2: Identify
- isReg: {T2,T6};
- isCit: {T2,T4};
- age: {T3,T4}; {T5,T6};

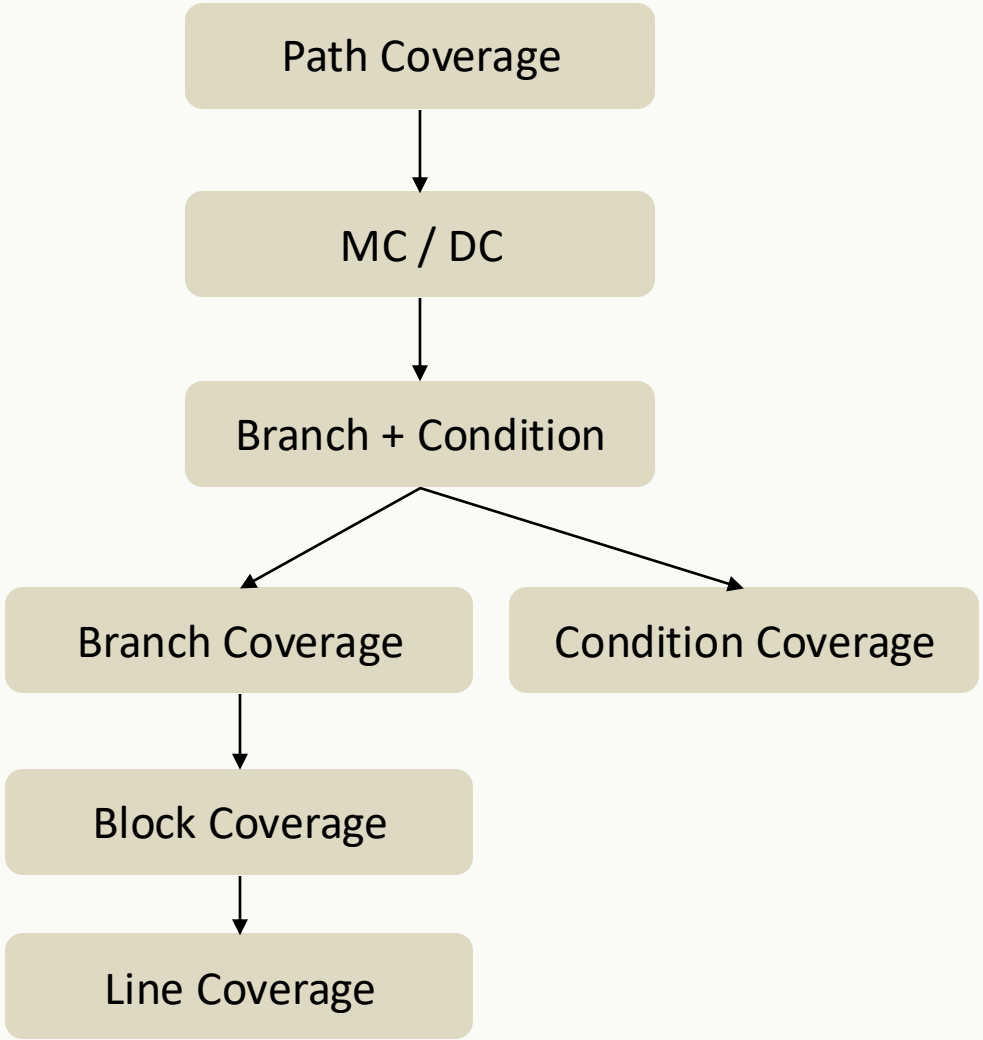
  {T7,T8}
- Final: {T2, T4, T5, T6}

Test	isRegistered	isCitizen	age >= 18	Decision
T1	FALSE	FALSE	FALSE	FALSE
T2	FALSE	FALSE	TRUE	FALSE
T3	FALSE	TRUE	FALSE	FALSE
T4	FALSE	TRUE	TRUE	TRUE
T5	TRUE	FALSE	FALSE	FALSE
T6	TRUE	FALSE	TRUE	TRUE
T7	TRUE	TRUE	FALSE	FALSE
T8	TRUE	TRUE	TRUE	TRUE

© Francisco Oliveira Neto - 2018

Subsumed Coverage Criteria

- **Subsumed criteria** are those that have full coverage whenever another criteria also has full coverage.
- Strategy **X subsumes strategy Y** if all elements that Y exercises are also exercised by X.
- Path coverage **subsumes all**.
- **Why not always do path coverage?**

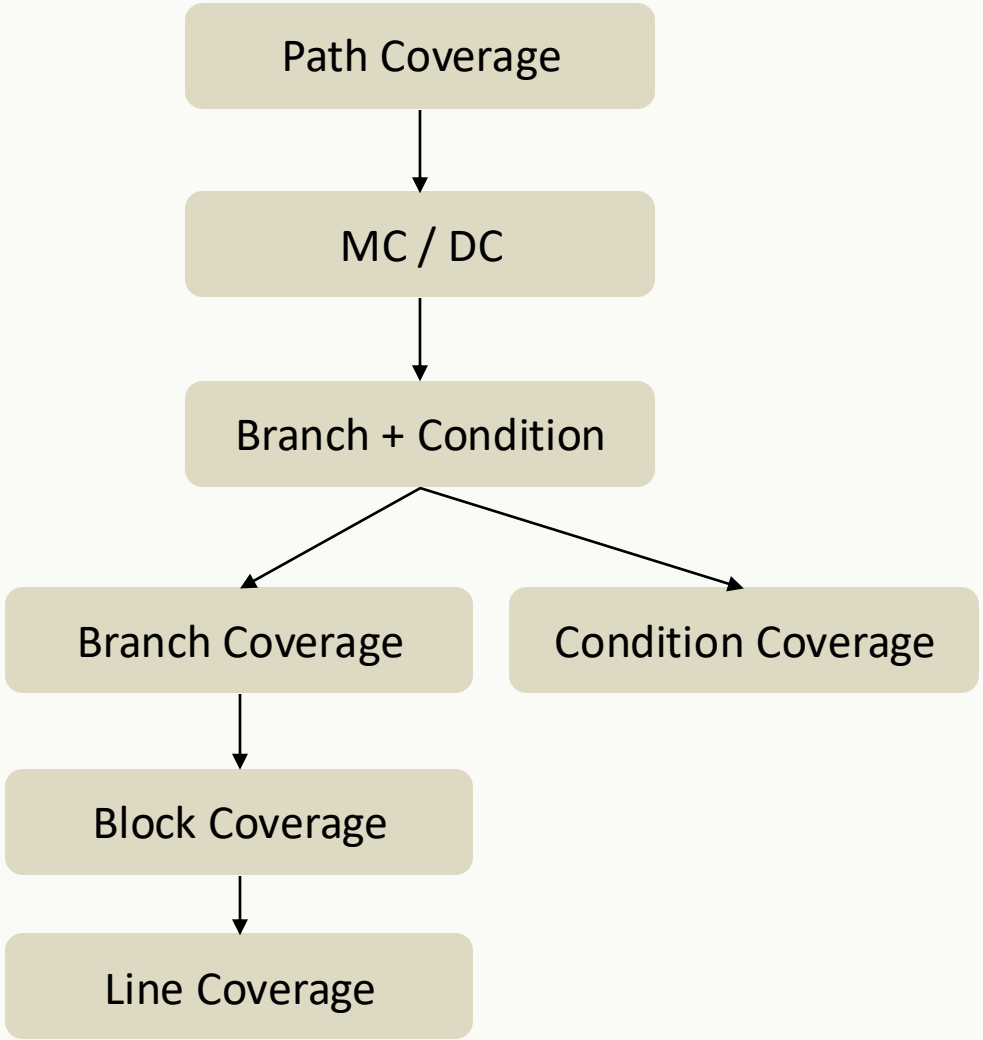


© Francisco Oliveira Neto - 2018

Subsumed Coverage Criteria

- **Subsumed criteria** are those that have full coverage whenever another criteria also has full coverage.
- Strateg
  elemen
  by X.
- Path cov
- **Why not always do path coverage?**

Because covering **all paths** would require a very **large number of tests**, such that it becomes **prohibitive**.



© Francisco Oliveira Neto - 2018

Limitations with Structural approaches

- Sometimes, there are **impossible combinations** of conditions or **unreachable statements**.
  - Examples: `while(true); if( x < 18 && x > 100);`

- Consequently, establishing 100% coverage as the acceptable threshold is risky. Sometimes we need to **settle for 80-95% coverage**.
- Choice of coverage criteria is **context dependent**:
  - Stronger criteria (MCDC) are needed for safety critical, but require lots of effort.
  - Less critical domain may settle for branch or even block coverage.

© Francisco Oliveira Neto - 2018

Structural-based vs. Specification-based

- Structural testing is **more objective** because they rely on information from the source code, instead of subjective opinions from humans.
- Structural testing can **cover gaps in the code** that are not necessarily captured by the specification alone (error-handling code for exceptions, utility functions, etc.)
- Structural testing cannot target the specification directly. By focusing on the implementation testers **can miss important functionalities** to test. 100% code coverage can reveal faults, but does not convey validation.
- Structural testing should **complement specification-based testing**.

© Francisco Oliveira Neto - 2018